

Anotações — Aula 1: Introdução ao Machine Learning

Disciplina: Machine Learning — Pós-Graduação Ciência de Dados e Big Data / Estatística

Professor: Murilo Afonso Robiati Bigoto (PUC Minas)

Dataset usado: dados_vlor_imovel.csv (n_comodos × valor) e dados_compra_casa.csv

Objetivo do Curso

O professor deixou claro desde o início: **o objetivo não é formar experts em ML**, mas sim fornecer **ferramentas conceituais e técnicas** que permitam ao aluno:

- Entender e navegar pelo universo de ML quando surgir no dia a dia
- Contribuir com conceitos em discussões técnicas
- Ter capacidade mínima de desenvolvimento e interpretação de modelos

***Analogia:** Pense no curso como um curso de culinária introdutório. Você não vai virar chef, mas vai saber fritar um ovo, seguir uma receita e entender o cardápio de um restaurante estrelado.*

Ementa do Curso (Visão Geral)

Unidade	Tema
1	Processo de Aprendizado de Máquina
2	Feature Engineering (Engenharia de Atributos)
3	Aprendizado Supervisionado e Não-Supervisionado
4	Modelos Ensemble
5	Métricas e Avaliação de Modelos
6	Explicabilidade, Interpretação e IA Generativa

O que é Machine Learning?

Machine Learning é um subcampo da Inteligência Artificial onde **ensinamos computadores a aprender a partir de dados**, sem programar explicitamente cada regra.

***Analogia:** Em vez de dar ao computador um manual de regras (ex: “se a casa tem mais de 3 quartos, vale mais de R\$500k”), você mostra **exemplos** de casas com preços e deixa o modelo descobrir as regras sozinho — como um detetive que resolve o crime a partir das pistas.*

Tipos de Aprendizado

1. Aprendizado Supervisionado

O modelo aprende com dados **rotulados** — ou seja, você sabe a resposta certa.

- **Regressão:** a saída é um valor contínuo (ex: preço de um imóvel)
- **Classificação:** a saída é uma categoria (ex: comprar ou não comprar a casa)

2. Aprendizado Não-Supervisionado

O modelo aprende com dados **sem rótulos** — ele descobre padrões sozinho.

- Ex: Agrupamento de clientes por perfil (clustering)

Analogia Supervisionado vs Não-Supervisionado:

Supervisionado = aprender matemática com um professor que corrige seus exercícios

Não-Supervisionado = organizar sua gaveta de meias sozinho — você cria as categorias

Conceito Central: X e Y

Todo problema de ML supervisionado tem essa estrutura:

Símbolo	Nome	O que é
X	Features / Variáveis de entrada	O que você dá de informação ao modelo

Y	Target / Rótulo / Variável alvo	O que você quer que o modelo preveja
---	---------------------------------	--------------------------------------

Exemplo da aula:

- X = número de cômodos da casa
- Y = valor do imóvel

Viés e Variância (Bias-Variance Tradeoff)

Este é um dos conceitos mais importantes em ML. É o equilíbrio entre um modelo **muito simples** e um **muito complexo**.

Situação	Nome	Problema
Modelo muito simples	Alto Viés (Underfitting)	Não aprende nem os dados de treino
Modelo muito complexo	Alta Variância (Overfitting)	Decora os dados de treino, vai mal nos novos
Equilíbrio ideal	Bom ajuste	Generaliza bem para dados novos

Analogia do estudante:

- **Underfitting** = aluno que não estudou nada e chuta tudo
- **Overfitting** = aluno que decorou o gabarito mas não entendeu o conteúdo — vai mal em qualquer prova diferente
- **Bom ajuste** = aluno que entendeu o conteúdo e se sai bem em qualquer prova

Pipeline Básico de ML (Regressão Linear)

Bibliotecas usadas

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
```

Por que essas bibliotecas?

- `numpy` : operações matemáticas e matrizes (o “motor” por baixo de tudo)
- `pandas` : manipulação de dados em tabelas (DataFrames)
- `matplotlib` : plotagem de gráficos
- `sklearn` (scikit-learn): repositório de algoritmos de ML prontos para uso — como uma “caixa de ferramentas” com modelos, métricas, pré-processadores, etc.

Passo 1 — Leitura dos dados

```
df = pd.read_csv('dados_vlor_imovel.csv')
print(df.head())
```

O arquivo `dados_vlor_imovel.csv` tem 20 linhas e 2 colunas:

- `n_comodos` (int): número de cômodos da casa
- `valor` (float): valor do imóvel

Passo 2 — Separar X e Y

```
X = df[['n_comodos']] # Feature de entrada (2D, necessário para sklearn)
Y = df['valor']       # Target que queremos prever
```

Detalhe técnico: o `X` precisa ser 2D (lista de listas) para o `sklearn`. Por isso usamos `[['n_comodos']]` com colchetes duplos. O `Y` pode ser 1D.

Passo 3 — Divisão Treino/Teste

```
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.3, random_state=42
)
```

- **70% dos dados** : treinamento (o modelo aprende com eles)
- **30% dos dados** : teste (dados que o modelo **nunca viu**, usados para avaliar qualidade)

- `random_state=42` : garante reprodutibilidade (o “embaralhamento” será sempre o mesmo)

***Analogia:** É como estudar com 70% das questões de um simulado e usar os 30% restantes como “prova surpresa” para ver se realmente aprendeu — e não apenas decorou.*

Passo 4 — Treinar o modelo (`.fit()`)

```
model = LinearRegression()    # Instancia (cria) o modelo
model.fit(X_train, Y_train)    # Treina o modelo com os dados de treino
```

- `LinearRegression()` : cria o objeto do modelo (ainda não sabe nada)
- `.fit(X_train, Y_train)` : **aqui acontece o aprendizado** — o modelo ajusta seus parâmetros internos para minimizar o erro entre o que prevê e o que é real

***Analogia:** `.fit()` é como a sessão de estudos. Você passa os exercícios (`X_train`) e as respostas (`Y_train`) para o modelo e ele aprende o padrão.*

Passo 5 — Fazer previsões (`.predict()`)

```
Y_pred_test = model.predict(X_test) # Predição nos dados de TESTE
Y_pred_train = model.predict(X_train) # Predição nos dados de TREINO (comparação di
```

O modelo usa o que aprendeu para prever o `Y` dado um `X` **que nunca viu**.

Passo 6 — Visualizar os resultados

```
import numpy as np

X_range = np.linspace(X.min() - 1, X.max() + 1, 100).reshape(-1, 1)
Y_range_pred = model.predict(X_range)

plt.figure(figsize=(10, 6))

# Dados de treino (azul)
plt.scatter(X_train, Y_train, color='blue', label='Dados de Treino (Reais)', alpha=0.7)
```

```
# Dados de teste (verde)
plt.scatter(X_test, Y_test, color='green', label='Dados de Teste (Reais)', alpha=0.7)

# Linha de regressão (vermelha)
plt.plot(X_range, Y_range_pred, color='red', linestyle='--', label='Linha de Regressão (Modelo)')

plt.title('Regressão Linear: Dados de Treino e Teste com Linha de Regressão')
plt.xlabel('Quantidade de Cômodos')
plt.ylabel('Valor da Casa')
plt.legend()
plt.grid(True)
plt.show()
```

Leitura do gráfico:

- **Pontos azuis** = dados usados no treino
- **Pontos verdes** = dados de teste (nunca vistos pelo modelo)
- **Linha vermelha** = predição do modelo

Quanto mais perto a linha vermelha estiver dos pontos verdes, **melhor o modelo generalizou**.

Exemplo de Classificação: Árvore de Decisão

O professor também mencionou um exemplo de **classificação** usando o arquivo `dados_compra_casa.csv` (50 linhas, com colunas: `Valor_Casa`, `Bairro`, `Num_Cômodos`, `Comprar`).

```
import pandas as pd
import numpy as np
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# Carregar dados
```

```
df_class = pd.read_csv('dados_compra_casa.csv')
print(df_class.head())
```

A diferença para a regressão: aqui o *Y* é uma categoria (*Comprar* = Sim/Não), não um número contínuo.

Scikit-learn como “Repositório de Modelos”

O professor recomendou explorar o site oficial: <https://scikit-learn.org>

Lá você encontra:

- Lista de modelos de classificação, regressão e clustering
- Como importar cada modelo
- Explicação conceitual do algoritmo
- Complexidade computacional
- Exemplos de código

Padrão de uso do sklearn (sempre o mesmo!):

```
from sklearn.<módulo> import <Modelo>

model = <Modelo>()      # 1. Instanciar
model.fit(X_train, Y_train) # 2. Treinar
Y_pred = model.predict(X_test) # 3. Prever
```

Resumo do Pipeline Completo

Dados brutos

Separar X (features) e Y (target)

Dividir em Treino (70%) e Teste (30%)

Escolher e Instanciar o Modelo (sklearn)

Treinar: `model.fit(X_train, Y_train)`

```
Prever: model.predict(X_test)
```

Avaliar / Visualizar

Pontos-Chave para Fixar

1. **X = entrada, Y = saída** — separar sempre antes de treinar
2. **Treino ≠ Teste** — nunca use os dados de teste no treinamento
3. **.fit() = aprendizado, .predict() = aplicação do modelo**
4. O sklearn é um “repositório” — quase todo algoritmo segue o mesmo padrão de uso
5. Overfitting e underfitting são o principal desafio do ML — o equilíbrio (bias-variance tradeoff) é o coração da modelagem

Referências

- [Scikit-learn — User Guide](#)
- Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*
- Müller & Guido. *Introdução ao Machine Learning com Python*
- [Notebook da Aula 1 — Aula1.ipynb] (material fornecido pelo professor)